Group Project: Selection & Strategy

Group Project Specification Due: February 12th

1. Project Options

EcoRoute: Campus Sustainability & Carbon Tracking System

- **Target Users:** University of Glasgow students and staff.
- **Core Functions:**
  - Users log daily commute methods (walking, cycling, bus, subway).
  - System calculates carbon savings based on input and generates personal/department leaderboards.
  - **External API Integration:** Call Google Maps API to calculate distances; or call OpenWeather API (to encourage walking when the weather is good).
- **DB Complexity:** User table, activity logs, transport emission coefficients, challenges/badges.
- **Recommendation:** Aligns with the "Sustainability" trend; the UI can be very clean and modern.

SwapSkill: Campus Skill Sharing & Peer-to-Peer Platform

- **Target Users:** Students wanting to learn new skills (coding, instruments, languages).
- **Core Functions:**
  - Users post skills they can teach and skills they want to learn.
  - Matching system: Recommends partners based on tags.
  - Booking system: Schedule mutual help sessions online and leave reviews for each other.
  - **External API Integration:** Integrate Google Calendar API (to auto-sync booking times).
- **DB Complexity:** User table (distinguishing Mentor/Mentee roles), skill categories, booking logs, reviews.
- **Recommendation:** The ER diagram will involve many-to-many (N:N) relationships, perfect for demonstrating database design skills.

🏠 *UniNest: Glasgow Student Housing Review & Evaluation System*

- **Target Users:** International students looking for housing in Glasgow and struggling with bad agencies.
- **Core Functions:**
  - Search for apartments by postcode; view real ratings and reviews (photos + text) from previous tenants.
  - Comparison feature: Compare cost-effectiveness, heating conditions, and landlord reliability.
  - **External API Integration:** Integrate Mapbox or Google Maps to show apartment locations; integrate UK Postcode API to validate addresses.
- **DB Complexity:** User table, property info, review table (including photo paths), agency blacklist.
- **Recommendation:** High pain point, high value; very easy to write compelling User Stories. **(Top Pick for Logic Stability)**.

🥑 *WasteNot: Community/Dorm Food Sharing & Recipe Recommendation*

- **Target Users:** Students wanting to save money and reduce food waste.
- **Core Functions:**
  - "Fridge Management": Users input food items nearing expiry.
  - "Food Sharing": Users post surplus food for neighbors to collect.
  - **External API Integration:** Call Spoonacular API (to auto-generate recipes based on current ingredients).
- **DB Complexity:** User table, food inventory, sharing posts, favorite recipes.
- **Recommendation:** Excellent logical loop involving "Data Input -> API Processing -> Result Feedback."

🎸 *GigsGlass: Glasgow Underground Music & Event Finder*

- **Target Users:** Music lovers looking for niche/local gig information.
- **Core Functions:**
  - Users follow different venues (pubs, live houses) to get event notifications.
  - Users upload their own gig photos and reviews.
  - Personalized Calendar: Save upcoming gigs; system sends email reminders before they start.
  - **External API Integration:** Call Ticketmaster API for mainstream data; call Email Service

API (e.g., SendGrid) for reminders.

- **DB Complexity:** User table, venue table, event table, user-favorite mapping.
- **Recommendation:** Glasgow is a UNESCO City of Music; this idea is very local and will stand out during the presentation.

💡 2. Tutor's "Get an A" Tips

1. **Which one is best?** If you want the most stable logic, pick **3 (UniNest)** or **2 (SwapSkill)**. Their ER relationships are very clear, and user roles (Tenant/Visitor or Mentor/Mentee) are distinct, allowing for great access control demonstrations.
2. **Regarding APIs:** No matter which you pick, you **must** include an external API in your architecture diagram (e.g., the Map API for UniNest). This is a "free" 2 marks, but it must be clearly shown.
3. **Regarding Data Input:** Ensure "User input data is re-utilized by the system." For example, in WasteNot, the user's "Potatoes" input is used to generate "Potato Recipes." This is a major scoring point.

🛠️ 3. General Technical Stack (Django-based)

- **Backend:** Django (Python) + Django REST Framework (if doing a decoupled frontend).
- **Database:** SQLite (Development) / PostgreSQL or MySQL (Production). **Note:** ER Diagram must reflect Django's `OneToOne`, `ForeignKey`, and `ManyToManyField` logic.
- **Frontend:**
  - **Option A (Traditional):** Django Templates + Bootstrap/Tailwind CSS + JavaScript (Vanilla/jQuery). **(Recommended: Faster results, fits course vibes)**.
  - **Option B (Modern):** React/Vue + Axios (communicating via API).
- **External APIs:** Google Maps API, Mailgun/SendGrid (Email), Spoonacular (Recipes), OpenWeather.

👥 4. 3-Member Task Allocation (General Model)

Member	Role	Design Spec Modules Responsible For	Key Assessment Points
Member	**PM /**	Overview, MoSCoW Requirements (6-10	Logical consistency,

A	**Copywriter**	items), Accessibility Plan, PPT Assembly, AI Statement.	WCAG 2.2 application.
Member B	**Architect / Backend**	System Architecture Diagram, **ER Diagram (Chen Notation)**, Data Dictionary.	ER diagram standards, N:N relationship handling.
Member C	**UI Designer / Frontend**	**Site Map**, **Wireframes**.	Interaction logic, visual coverage of "Must" requirements.

5. Specific Detail & Logic Refinement

- **UniNest (Logic-focused):**
  - Django: Use built-in Auth for Student/Admin roles.
  - DB: Review table must link to `User` (FK) and `Property` (FK).
- **SwapSkill (DB-focused):**
  - Django: Use `ManyToManyField` for "Liked Skills" and "Proficient Skills".
  - DB: ER must show the "Booking" intermediary entity.
- **WasteNot (API-focused):**
  - Django: Expiry logic (calculating `expiry_date`).
  - Architecture: Show how Django requests external APIs and processes JSON data.

6. Our "Action Plan" for an A

1. **Tonight (Step 1):** Select one idea and confirm our roles (Member A/B/C).
2. **Tomorrow (Step 2):**
   a. **A** starts writing 8 User Stories (must include Login, Input, and Save to DB).
   b. **B** starts the draft ER diagram based on stories.
   c. **C** starts the Site Map based on stories.
3. **Day After (Step 3): B** and **C** sync: Does every feature in the Site Map have a corresponding table in the ER diagram?
4. **Final (Step 4): C** polishes wireframes (Figma/Draw.io); **A** writes the crucial **Accessibility Plan**.

⚠️ **Note:** In our PDF, we must emphasize **Django's** role—specifically label the "Django Web

Server" and "Django ORM" in the architecture diagram.